

# Characterizing High Bandwidth Flash for LLM Serving

Zack Yu<sup>✉</sup>  
zack.yu@berkeley.edu  
University of California,  
Berkeley  
Berkeley, California, USA

Chloe Wong  
chloe.wong@berkeley.edu  
University of California,  
Berkeley  
Berkeley, California, USA

Coleman Hooper<sup>✉</sup>  
chooper@berkeley.edu  
University of California,  
Berkeley  
Berkeley, California, USA

Minjae Lee  
minjae.lee@furiosa.ai  
FuriosaAI  
Seoul, South Korea

Wonjun Kang  
kangwj1995@furiosa.ai  
FuriosaAI  
Seoul, South Korea

Youngjin Cho  
youngjin.cho@furiosa.ai  
FuriosaAI  
Seoul, South Korea

Michael W. Mahoney  
mahoneymw@stat.berkeley.edu  
University of California,  
Berkeley; ICSI; LBNL  
Berkeley, California, USA

Yakun Sophia Shao  
ysshao@berkeley.edu  
University of California,  
Berkeley  
Berkeley, California, USA

Kurt Keutzer  
keutzer@berkeley.edu  
University of California,  
Berkeley  
Berkeley, California, USA

Amir Gholami<sup>✉</sup>  
amirgh@berkeley.edu  
University of California,  
Berkeley; ICSI  
Berkeley, California, USA

## Abstract

Large language model (LLM) serving requires substantial memory to store model weights and KV caches. As models grow larger and contexts become longer, memory capacity and bandwidth increasingly become bottlenecks for serving performance. Agentic workloads compound this pressure through repeated interactions over growing contexts, making it increasingly important to retain KV state for reuse. High-bandwidth flash (HBF) offers a way to expand accelerator memory capacity for large language model (LLM) serving, but its access costs and limited write endurance complicate its use. We evaluate HBF for high-throughput agentic serving across system design and scheduling choices to understand when additional capacity improves serving performance and energy efficiency. We introduce an HBM-HBF-host hierarchical storage system and buffered cache-aware scheduling, and use trace-driven simulations to analyze their effects on performance, energy consumption, and HBF write lifetime. Across the evaluated workloads, the fastest HBF-augmented systems reduce completion time by 36.1–87.0% relative to HBM-only systems. Modeled energy savings reach 55.8%, although HBF increases energy consumption on some light workloads. Buffered cache-aware scheduling extends estimated HBF write lifetime from 4.77 to 14.82 years in the evaluated configuration. These results demonstrate the importance of coordinating data placement and scheduling to improve serving efficiency while sustaining a practical HBF write lifetime.

## Keywords

High-bandwidth flash, LLM serving, LLM inference, KV cache management, Hierarchical KV cache, AI infrastructure, LLM serving simulation

## 1 Introduction

LLM serving places competing demands on accelerator memory. Model weights occupy substantial capacity, while KV state grows

with context length and the number of active requests. Accelerator compute throughput has historically grown faster than memory bandwidth, widening the gap between how quickly data can be processed and how quickly it can be supplied [5]. Larger batches can improve weight reuse and kernel utilization, but require more memory for active KV state [10, 25]. Retaining cached prefixes adds another demand on capacity, yet can avoid computation when later requests reuse them [36]. A serving system must balance these uses of memory to sustain efficient execution.

Agentic serving makes this balance particularly important. Repeated model calls can extend a shared context with new instructions, intermediate results, and tool outputs [9, 18]. The KV state of a completed request may therefore remain useful for a later call. Admitting more requests can displace this state before it is reused, requiring host reloads or repeated prefill [13]. Long contexts increase both the space needed to retain reusable prefixes and the cost of recovering them. Agentic workloads thus motivate studying memory capacity together with cache retention, rather than treating capacity only as a way to increase batch size.

High-bandwidth flash (HBF) offers a way to expand device memory and reduce dependence on host storage [6, 24]. Its capacity can accommodate model weights, larger active KV caches, and more reusable prefixes. However, HBF has different costs from HBM. In the configuration studied here, HBF reads consume more energy than HBM reads, writes are much slower, and repeated writes consume finite program/erase endurance [6, 19]. Simply adding capacity does not ensure that a serving policy uses it efficiently. Effective use of HBF requires deciding which data belongs in each tier and when admitting more work is worth the resulting cache pressure.

We compare Sandisk’s shared-site architecture, where HBM and HBF share a limited number of sites around the accelerator, with the H<sup>3</sup> architecture, where HBM occupies all sites and HBF is chained behind HBM [6, 24].

We introduce an HBM-HBF-host hierarchical storage system that manages weight placement, KV residency, and cache movement across HBM, HBF, host DRAM, and SSD. New KV state and reloaded prefixes enter HBM, while less recently used cache can move to lower tiers as capacity becomes scarce. We complement this hierarchical storage system with buffered cache-aware scheduling, which uses an admission buffer to leave room for decode growth and reduces pressure to evict reusable caches.

We target high-throughput serving settings like synthetic-data generation, asynchronous agent workflows, and reinforcement learning rollouts, where many independent sessions can be processed concurrently. Our focus is on settings that prioritize overall workload completion time and energy efficiency and can tolerate increased per-request latency. In this setting, additional capacity translates directly into larger batches and more retained KV state. We evaluate the hierarchy and scheduling policy using a request-level simulator with kernel latency models fitted to GPU measurements. The simulator tracks scheduling, cache residency, data movement, and parallel execution while replaying multi-turn workloads for dense and sparse-MoE models. Our evaluation covers both the original traces and synthetically extended conversations, comparing completion time, modeled energy, cache reuse, and estimated write lifetime. Energy and completion-time decompositions explain how weight reuse, repeated prefill, KV accesses, and data transfers contribute to the observed benefits and costs. This paper makes three contributions:

- An HBM-HBF-host hierarchical storage system for LLM serving, with placement and data-movement policies that account for each tier’s capacity and access costs. On the 1× traces for Llama-3.1-405B and GLM-5.2, hybrid systems retaining weights in HBM reduce completion time by 46.0–56.3% and modeled energy by 14.4–24.1% relative to HBM-only baselines.
- Buffered cache-aware scheduling that preserves reusable KV state through admission control. In the scheduling experiment, reserving 10% headroom reduces completion time by 4.7% and extends estimated HBF write lifetime from 4.77 to 14.82 years relative to unbuffered cache-aware scheduling, under the modeled endurance assumptions.
- A trace-driven analysis of how workload characteristics, weight and expert placement, and parallelism affect HBF’s value for serving. The analysis identifies the different sources of savings in dense and sparse serving and shows how longer conversations change the performance and energy tradeoffs.

## 2 Related Work

*LLM serving and KV-cache management.* Orca, vLLM, and SGLang improve batching, KV-memory management, and prefix reuse [10, 34, 36], while FlexGen coordinates GPU, CPU, and disk resources for throughput-oriented inference [25]. Autellix and ThunderAgent schedule agent programs across model and tool calls [9, 18]; Continuum uses KV-cache time-to-live to balance retention against admission pressure [13]. HiCache, LMCACHE, and Mooncake support hierarchical or distributed KV storage and reuse [15, 23, 32], with CacheGen and CacheBlend addressing KV compression and

cached-context reuse [16, 33]. Our focus is coordinating these storage and scheduling decisions under HBF’s asymmetric access costs and finite write endurance.

*HBF and flash-based inference.* The Sandisk HBF roadmap and H<sup>3</sup> architecture motivate flash capacity near accelerators [6, 24]. Existing flash-based designs cover weight offloading, SSD-based KV swapping, computation in flash, and vector search [1, 7, 8, 35]. Concurrent HBF serving studies highlight the challenges of storing KV caches in HBF, adopting SSD-style KV backing or restricting HBF to static expert weights, while others identify endurance as a major barrier [14, 22, 27]. We address these challenges through hierarchical placement and buffered cache-aware scheduling to reduce cache churn and repeated writes.

*Simulation-based evaluation.* LLMservingSim 2.0 models heterogeneous and disaggregated serving infrastructure [3]; ASTRA-sim2.0 models distributed training systems [30]; and MemExplorer explores heterogeneous memory designs for agentic-inference NPUs [31]. Our request-level simulation examines how placement and admission decisions affect completion time, energy, and writes across an HBM-HBF-host hierarchy.

## 3 Methodology

### 3.1 Goal of Study

Our goal is to characterize how adding HBF improves LLM serving efficiency and what algorithms are necessary to obtain that improvement. We evaluate high-throughput serving through fixed-workload completion time and modeled energy, without imposing a per-request latency target. We also report cache-hit fractions, KV writes, and an endurance-based lifetime estimate. Comparisons hold the request workload and accelerator count fixed within each experiment. Completion time is the elapsed simulated time required to finish all requests, including arrivals, inter-turn gaps, computation, communication, and cache management. We report mean time per output token (TPOT) to expose the accompanying latency tradeoffs.

### 3.2 Simulation Method

*Request execution and cache state.* The simulator tracks request arrivals and simulates execution asynchronously across model replicas. Each step selects eligible work, performs required cache preparation, executes a forward pass, advances requests, and accounts for subsequent cache movement. Mixed batches can contain both prefill and decode work. Chunked prefill splits prompt processing across steps under a shared prefill-token budget, allowing prefill chunks to execute alongside decode requests with reasonable latency. Decode tokens do not consume this budget. Requests in a session execute in order, and the next turn becomes eligible after the preceding turn completes and its recorded gap elapses. The simulator tracks residency separately in HBM, HBF, host DRAM, and SSD. Prefixes already on the device avoid host reload, prefixes on the host incur transfer service, and unavailable prefixes are computed again. KV state continues to grow during decoding, so active requests can exhaust HBM even without new admissions. We model this decode overflow and the resulting cache eviction and offload costs, including any reloads needed before subsequent execution.

The simulator operates at request-level, tracking the number of tokens stored in each memory tier rather than individual token positions. We assume that token placement within each tier follows the specified policy.

**Kernel Modeling.** We calibrate kernel latency models using GPU measurements on NVIDIA H100 and B200. GEMM-style kernels use a roofline model combining launch overhead with compute and memory service time; grouped GEMM accounts for routed tokens and active-expert weights. HBM and HBF service times are added for  $H^3$  and combined by their maximum for shared-site GEMM and decode attention.

Attention is modeled separately for prefill and decode. Prefill attention latency depends on the number of query-key pairs evaluated and the attention kernel implementation, while decode uses a fitted memory-bandwidth utilization model. Sparse MLA extrapolates from dense MLA profiles by limiting attention work to the selected keys. The indexer accounts separately for projections, scoring, top- $k$  selection, and index writes, with scoring and selection latency fitted to query count and context length. Indexer scoring uses an FP8 timing proxy while storage and memory-energy accounting remain 16-bit; top- $k$  energy is not calibrated. Mixed-batch and HBF performance are modeled extrapolations from the measured kernels.

**Energy Modeling.** Energy is accumulated from arithmetic operations, modeled memory accesses, and communication:

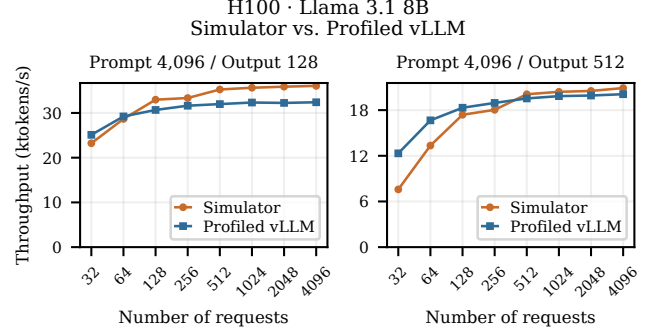
$$E = \sum_j N_j e_j + \sum_m 8(R_m e_m^r + W_m e_m^w) + E_{\text{comm}}, \quad (1)$$

where  $j$  indexes the arithmetic operations in Table 2,  $N_j$  is the operation count, and  $e_j$  is the corresponding energy per operation. For memory tier  $m$ ,  $R_m$  and  $W_m$  are bytes read and written, and  $e_m^r$  and  $e_m^w$  are read and write energy per bit.  $E_{\text{comm}}$  is the energy consumed by data transfers over communication links. We estimate energy from operation counts and memory and communication traffic, without adding static power consumption. Faster kernels alone therefore do not reduce modeled energy; savings come from fewer operations, less data movement, or accesses to lower-energy memory tiers.

**Parallelism Modeling.** We model TP [26], EP [12], and data-parallel attention (DPA) [4]. With DPA enabled, each attention DP rank handles its own requests and KV state, including full MLA QKVO projections and indexer work. FFN/MoE execution uses the combined rank batches and the configured TP/EP layout. Attention-rank latency is reduced by a maximum at the modeled synchronization before the shared FFN/communication phase. For models using IndexCache [2], we separately model full layers, which execute the indexer, and shared layers, which reuse an earlier full layer’s sparse-token selection. For each layer category  $g$ , let  $L_g$  denote the number of layers in that category. The forward latency is approximated as follows:

$$T_{\text{forward}} = \sum_g L_g \left( \max_r T_{g,r}^{\text{attn}} + T_g^{\text{FFN}} + T_g^{\text{comm}} \right). \quad (2)$$

Energy sums activity across ranks. Independent replicas maintain separate clocks and share modeled host resources; total completion time is the latest replica completion.



**Figure 1: Simulator versus profiled vLLM on H100: total input-plus-output token throughput. Simulated request lengths follow lognormal distributions with  $\sigma = 0.5$ ; host KV offload is disabled. Each 4,096-request vLLM run includes 20 failed requests.**

**Communication Modeling.** Independent PCIe links can transfer data concurrently, while transfers sharing host DRAM or SSD compete for bandwidth. After startup, let  $r_i$  be the fraction of transfer  $i$  completed per second. Let  $s_{i,L}$ ,  $s_{i,D}$ , and  $s_{i,S}$  be its standalone bandwidth service times for the local PCIe/device-memory path, host DRAM, and SSD. The local time is the maximum of link and device-memory service times; host times include reads and writes required for staging. For the active transfers in a scheduling call, rates satisfy

$$\begin{aligned} 0 &\leq r_i, & r_i s_{i,L} &\leq 1, \\ \sum_i r_i s_{i,D} &\leq A_D(t), \\ \sum_i r_i s_{i,S} &\leq A_S(t), \end{aligned} \quad (3)$$

where  $A_D(t), A_S(t) \in [0, 1]$  are the bandwidth fractions available after earlier reservations. The simulator allocates transfer rates subject to these constraints and updates them as transfers start or finish or available bandwidth changes. Dependent transfers execute sequentially, and forward execution waits for the required cache data. GPU collectives include startup overhead plus the maximum of network transfer time and HBM buffer access time.

**Comparison with vLLM.** We compare our HBM-only simulator with profiled vLLM measurements [10] for Llama-3.1-8B on one NVIDIA H100 GPU. Figure 1 sweeps 32–4,096 single-turn requests, with a mean input length of 4,096 tokens and mean output lengths of 128 or 512 tokens in the simulator. The simulator captures the increase and saturation of throughput, with larger deviations at low request counts for the longer outputs.

### 3.3 Simulation Trace

The workload is derived from LMCACHE multi-turn agentic sessions [17], containing 24,880 turns in 767 sessions. We preserve each turn’s input length, output length, and inter-turn gap, using model-specific tokenization for input lengths. Output lengths and gaps are retained from the workload. The 25k-turn experiments contain 771 sessions; the 100k-turn experiments contain 3,080 sessions. To reach these counts, we replicate the original conversations

as independent sessions, preserving each turn’s input and output lengths. We retain only the required initial turns of the final session to reach the target request count. Replication increases the number of requests without extending their contexts.

We also construct a  $5\times$  trace by concatenating five copies of each session, preserving output lengths and within-copy gaps, with zero gaps between copies. Each appended copy’s input lengths are offset by the preceding copy’s final input-plus-output length. This yields 500k turns across the same 3,080 sessions and increases mean input length from 22,558 to 89,125 tokens.

*Sparse-attention lookup.* We estimate selected-token placement from offline GLM sparse-selection profiles. At a fixed reference context of 32,768 tokens, we measure the expected number  $f(k)$  of selected tokens in the most recent  $k$  positions and use piecewise-linear interpolation between sampled window sizes. There are at most  $s = 2,048$  selected tokens per query. For runtime context  $x$  and  $k$  HBM-resident KV tokens, let  $s_x = \min(x, s)$ . The model uses

$$h(x, k) = \text{clip}(f(k), \max(0, s_x - (x - k)), \min(k, s_x)), \quad (4)$$

$$h_{\text{HBF}}(x, k) = s_x - h(x, k).$$

This preserves the available token counts in both tiers.

*Expert lookup.* We profiled GLM-5.2’s expert routing on two sessions from the LMCACHE trace. One serve as training set that determines expert-copy allocation and HBM placement, while the other serve as test set to estimate how many expert copies are accessed in each batch.

## 4 System Overview

### 4.1 $H^3$ versus Sandisk’s Shared-Site Design

We consider two architectures shown in Figure 2. The Sandisk shared-site design places HBM and HBF around the accelerator’s limited space that can attach HBM/HBF stacks. The illustrated configuration divides eight positions into four HBM and four HBF stacks.  $H^3$  design instead connects HBF behind HBM, retaining eight HBM stacks while adding eight HBF stacks. We assume HBM buffering can stage data arriving from HBF to fully hide latency. In this case, the  $H^3$  architecture can provide larger capacity and better bandwidth than the shared-site design.

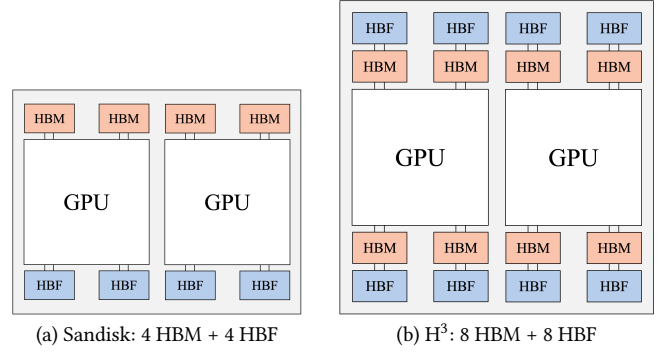
Let  $D_H$  and  $D_F$  be fixed amounts of data read from HBM and HBF, and let  $B$  be their equal per-stack read bandwidth. Suppose the shared-site design allocates  $m$  of its eight positions to HBM and the remaining  $8 - m$  to HBF, where  $0 < m < 8$ . With concurrent reads from the two tiers, its service time is

$$T_{\text{shared}} = \max\left(\frac{D_H}{mB}, \frac{D_F}{(8-m)B}\right). \quad (5)$$

Even if HBM and HBF reads are serialized in the  $H^3$  design, its read time is no greater than that of the shared-site design.

$$\begin{aligned} T_{8+8} &= \frac{D_H}{8B} + \frac{D_F}{8B} \\ &= \frac{m}{8} \frac{D_H}{mB} + \frac{8-m}{8} \frac{D_F}{(8-m)B} \\ &\leq T_{\text{shared}}. \end{aligned} \quad (6)$$

For GEMM and decode attention, we model HBM and HBF service sequentially for  $H^3$  and concurrently for the shared-site design. Data



**Figure 2: Two memory architectures we considered in this study. The sites refer to the space where HBM/HBF stacks can connect to the accelerator. HBM and HBF compete for shared sites in the first architecture, whereas HBM and HBF are connected sequentially in second architecture. We simulate both architectures.**

movement is pipelined across successive memory tiers, assuming overlap in transfer time. However, a chip that uses  $H^3$  design will be more expensive than a chip that uses the shared-site design due to more HBM/HBF used, therefore we evaluate both architectures in our simulation studies.

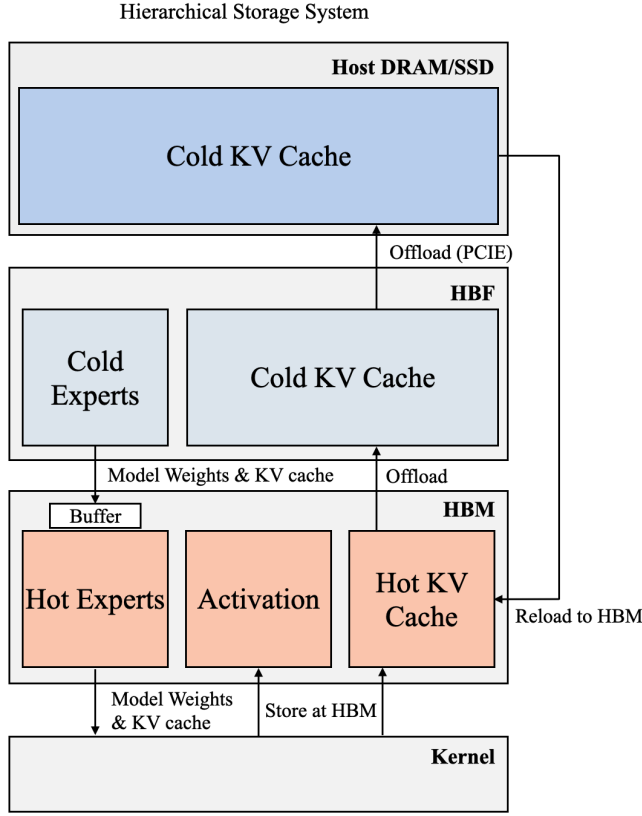
### 4.2 Hierarchical Storage System

Figure 3 shows the storage hierarchy. HBM contains frequently accessed weights and active data, while HBF expands the on-device working set. Host DRAM and SSD hold state that cannot remain on the accelerator. New KV state and reloaded prefixes are staged in HBM. As space becomes scarce, older KV state moves toward HBF and then the host. State needed by active requests is protected by reference locks subject to the simulator’s capacity and overflow handling.

*Weights and activations.* Weight placement is fixed for each evaluated configuration. Single-tier experiments place all weights in HBM or HBF; the split MoE configuration places frequently used experts in HBM and the remainder in HBF. Activation writes to HBM since repeatedly writing short-lived activations to flash would consume endurance rapidly.

*Cache eviction and offload.* We use request-level least-recently-used (LRU) ordering to select eligible cache entries when a memory tier runs out of space. HBM data is demoted to HBF or offloaded to host DRAM; DRAM data spills to SSD, and SSD data is discarded when necessary. Assuming LRU requests are less likely to be accessed than recently used request, this retains hotter cache on HBM and moves colder cache to the cold memory tiers. Completed-session caches remain available for reuse until evicted. Reference locks exclude active requests from ordinary LRU eviction, although their state can move from HBM to HBF when device capacity permits. Future turns reload offloaded prefixes and recompute missing ones.

*Index-first placement.* Sparse attention accesses only a selected subset of KV tokens, but the indexer access all index data across the



**Figure 3: Hierarchical placement of weights, activations, and KV state.** Host reload returns to HBM; colder device state can move through HBF to the host. The buffer drawn on the HBF read path is the latency hiding buffer of H<sup>3</sup> architecture

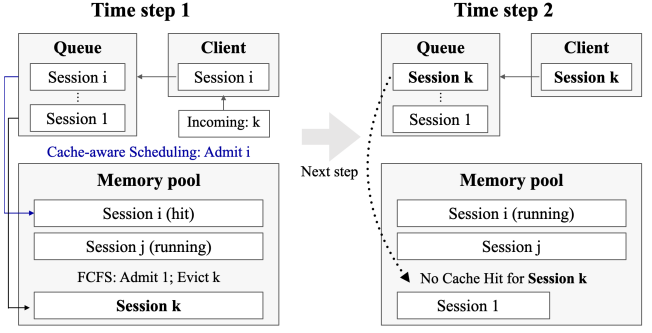
context. We therefore consider index as hotter data, and separate the residency of indices and attention KV for more fine-grained data movement. Under HBM pressure, eligible KV is demoted before indices. On reload, indices preferentially return to HBM, displacing eligible KV when necessary. Under HBF pressure, indices are offloaded to the host before attention KV, so that indices can be placed at HBM on reload. These priorities target to place the frequently accessed indices on HBM while placing more sparse KV in HBF.

### 4.3 Buffered Cache-aware Scheduling

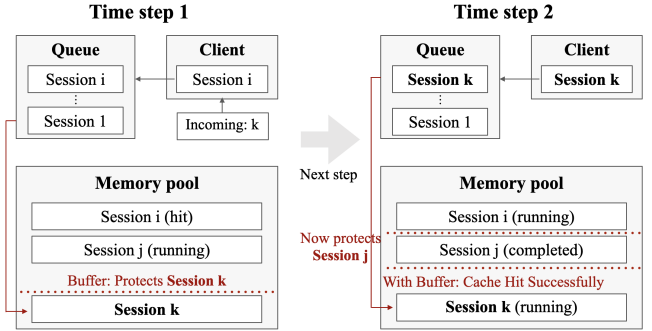
Cache-aware scheduling prioritizes waiting requests with more device-resident prefix tokens, and using queue order to resolve ties. This avoids unnecessary reload or prefill work when a reusable prefix is available. However, ordering alone does not control the amount of active state. If too many requests are admitted, their growing contexts can displace completed session’s KV cache before the next turn reuses it.

We add an admission fraction  $b \in [0, 1]$ . Let  $C$  be usable device KV/index capacity after resident weights,  $L$  the currently locked footprint, and  $A_i$  the footprint newly locked by admitting request  $i$ . Admission requires

$$L + A_i \leq bC. \quad (7)$$



**Figure 4: Vanilla cache-aware scheduling can lose a future cache hit.** At time step 1, the scheduler selects the cache hit from session  $i$ , then admits session 1 in queue order, evicting session  $k$ ’s cached prefix. When session  $k$  returns at time step 2, its prefix is no longer resident.



**Figure 5: Buffered cache-aware scheduling can preserve a future cache hit.** Delaying session 1 leaves session  $k$ ’s cached prefix resident for reuse at time step 2. Dotted lines illustrate admission headroom: session  $k$  is retained at time step 1, and session  $j$  completes and remains cached at time step 2.

A 10% buffer uses  $b = 0.9$ ; no buffer uses  $b = 1$ . In the implementation,  $A_i$  accounts for the incoming request’s full context footprint, including any cached prefix newly brought under an active lock. A hit saves computation or movement but does not make the prefix free of capacity cost. Completed-session cache is evictable and is excluded from  $L$  until re-admitted.

The threshold is checked at admission, not enforced as a fixed physical partition after every decode token. Active contexts can grow, and completed sessions can release locks while leaving cached data resident. The buffer limits aggregate admission pressure, thus protects residing KV cache, improving cache hit when the follow-up request gets scheduled by longest cache hit. Although the buffer constrains scheduling larger batches, the benefit of improved cache hit can result in higher throughput. Most importantly, the improved cache-reuse can significantly reduce writes, resolving the challenge of write lifetime when placing KV cache to HBF. As Table 4 shows, the buffer extends estimated HBF write lifetime to over a decade in both evaluated hybrid configurations. Moreover, if the P/E cycle of the released product is lower, or the user wants to be safer in using

**Table 1: Per-stack memory assumptions.** GB and TB use decimal units. HBF read energy is set to twice HBM’s following MemExplorer [31]; H<sup>3</sup> reports up to four times the power consumption [6]. Given limited published HBF write specifications, we use typical SSD write parameters and assume SLC endurance of 100,000 P/E cycles. HBM startup uses the upper end of the reported HBM4 read-latency range, with write startup assumed equal to read.

| Parameter                | HBM        | HBF          |
|--------------------------|------------|--------------|
| Capacity (GB)            | 24 [20]    | 375 [6]      |
| Read bandwidth (GB/s)    | 1,000 [11] | 1,000 [6]    |
| Write bandwidth (GB/s)   | 1,000 [11] | 8            |
| Read startup ( $\mu$ s)  | 0.1 [19]   | 20 [6]       |
| Write startup ( $\mu$ s) | 0.1 [19]   | 250          |
| Read energy (pJ/bit)     | 3.4 [21]   | 6.8 [31]     |
| Write energy (pJ/bit)    | 3.4 [21]   | 100          |
| Assumed P/E cycles       | —          | 100,000 [27] |

**Table 2: Arithmetic energy parameters at 4 nm, scaled from 7 nm by 0.546 [28, 29, 35].**

| Operation type $j$       | Energy $e_j$ (pJ/op) |
|--------------------------|----------------------|
| FP16 multiply-accumulate | 0.38                 |
| FP32 addition            | 0.22                 |
| FP32 multiplication      | 0.71                 |
| FP32 exponentiation      | 7.4                  |
| FP32 division            | 5.6                  |
| FP32 square root         | 11.1                 |

HBF, we can reduce the value of  $b$  to trade throughput for even less admission pressure. This can further increase the lifetime of HBF.

Figures 4 and 5 contrast unbuffered and buffered cache-aware admission. Preserving headroom can prevent useful cache from being displaced shortly before reuse. This may reduce instantaneous concurrency, yet improve workload completion time by avoiding later reloads, demotions, and repeated prefill.

## 5 Results

### 5.1 Experimental Setups

All experiments use the simulator’s B200 compute model and 16-bit weight/cache storage assumptions. Indexer latency uses an FP8 scoring proxy, as described in the kernel model. HBM-only devices contain eight 24-GB HBM stacks. Hybrid devices combine 24-GB HBM stacks with 375-GB HBF stacks: the H<sup>3</sup>-style configuration uses eight stacks of each type, while shared-site configurations divide eight stack sites between HBM and HBF. Stack allocations are specified for each experiment. Table 1 lists the device-memory parameters used in the study.

The host model provides 16 independent DDR5-7200 channels at 57.6 GB/s and 64 GB each, plus 32 SSDs at 8 GB/s and 1 TB each. GPU–host transfers use PCIe Gen5 x16 with approximately 63 GB/s per direction after line coding; GPU collectives use a 900-GB/s modeled NVLink endpoint bandwidth. Shared host bandwidth is

**Table 3: Workload and parallelism configuration.** TP/EP denotes tensor/expert parallelism with the stated group size; DPA denotes data-parallel attention. 2×TP8/EP8 uses two eight-GPU replicas. The 1× and 5× traces contain 100k and 500k turns; Qwen uses a separate 25k-turn workload.

| Model          | GPUs | Turns | Parallelism     |
|----------------|------|-------|-----------------|
| Qwen3-32B      | 1    | 25k   | TP1             |
| Llama-3.1-405B | 8    | 100k  | TP8             |
| GLM-5.2        | 16   | 100k  | TP16/EP16 + DPA |
| GLM-5.2        | 16   | 100k  | 2×TP8/EP8 + DPA |
| GLM-5.2        | 16   | 500k  | TP16/EP16 + DPA |
| GLM-5.2        | 16   | 500k  | 2×TP8/EP8 + DPA |

aggregated rather than multiplied by the number of simultaneous GPU transfers.

The 1× trace contains 100k turns with the original conversation lengths. For GLM, we construct the 5× trace by concatenating five copies of each session and accumulating context across copies, yielding 500k turns over the same sessions. The Qwen scheduling study separately uses 25k turns. Table 3 summarizes model sizes, workload lengths, and parallelism. The Qwen study varies scheduling with weights in HBM. The large-model studies use cache-aware scheduling with a 10% buffer, mixed batches, session-sticky routing, and KV offload. Llama does not use DPA; GLM uses DPA with either one TP16/EP16 replica or two TP8/EP8 replicas. All GLM points use index-first and the recency-only sparse lookup. All runs use a chunked-prefill budget of 2,048 tokens per scheduling engine per step, shared across prefill requests. For GLM DPA, each attention rank has its own scheduling engine. Each 1× or 5× workload is shared across replicas, not repeated independently on each replica.

Device and host token hit rates use total input tokens as a common denominator. Device includes HBM and HBF; host includes DRAM and SSD. These fractions are measured when requests are admitted for prefill. They are not the fraction of decode accesses served by HBM, and they need not sum to 100% because new or uncached input requires computation. Write counters report modeled KV traffic.

All sessions submit their first request at simulation time zero. Subsequent turns arrive after the preceding turn completes and the recorded inter-turn gap elapses. We measure the time and energy required to complete this fixed workload, without imposing an external request arrival rate.

In the result tables,  $X + Y$  denotes HBM+HBF stack counts per GPU. TPOT is the mean per-request time per output token after the first. Lower completion time, energy, and TPOT are better; bold entries in the Llama and GLM configuration tables mark the minimum in each metric column. All configurations in those two tables use a 10% scheduling buffer.

### 5.2 Scheduling and Write Endurance

Table 4 compares FCFS, cache-aware scheduling, and buffered cache-aware scheduling on Qwen. Reserving admission headroom preserves reusable KV state: on HBM alone, the buffer raises device hits from 6.61% to 56.79% and more than halves HBM KV writes

**Table 4: Qwen3-32B scheduling on one GPU, 25k turns, and HBM-resident weights. Shared-site 4 HBM + 4 HBF divides eight stack sites;  $H^3$  adds eight HBF stacks alongside eight HBM stacks. FCFS means first-come, first-served; Cache aware, 10% reserves 10% admission headroom (bold hybrid rows). Clock is workload completion time; KV writes are key-value-cache writes. Device/host hits are admitted input-token fractions served by HBM or HBF/host DRAM or SSD. Life estimates HBF write endurance via Equation 8; dashes mean no HBF.**

| Memory               | Scheduling              | Clock (s)        | Energy (MJ)  | HBM KV writes (TB) | HBF KV writes (TB) | Device hit (%) | Host hit (%) | Life (years) |
|----------------------|-------------------------|------------------|--------------|--------------------|--------------------|----------------|--------------|--------------|
| 8 HBM                | FCFS                    | 19,033.77        | 2.323        | 158.54             | 0.00               | 0.16           | 96.36        | —            |
| 8 HBM                | Cache aware             | 18,479.17        | 2.274        | 148.41             | 0.00               | 6.61           | 89.92        | —            |
| 8 HBM                | Cache aware, 10%        | 16,313.41        | 2.245        | 69.35              | 0.00               | 56.79          | 39.73        | —            |
| 4 HBM + 4 HBF        | FCFS                    | 23,228.20        | 2.983        | 8.23               | 147.15             | 7.30           | 89.23        | 0.75         |
| 4 HBM + 4 HBF        | Cache aware             | 20,821.73        | 2.849        | 7.91               | 93.00              | 41.72          | 54.80        | 1.06         |
| <b>4 HBM + 4 HBF</b> | <b>Cache aware, 10%</b> | <b>16,614.28</b> | <b>2.703</b> | <b>6.89</b>        | <b>7.48</b>        | <b>96.06</b>   | <b>0.47</b>  | <b>10.57</b> |
| 8 HBM + 8 HBF        | FCFS                    | 13,552.76        | 2.758        | 9.16               | 106.79             | 32.84          | 63.69        | 1.21         |
| 8 HBM + 8 HBF        | Cache aware             | 10,930.78        | 2.659        | 7.32               | 21.81              | 86.87          | 9.66         | 4.77         |
| <b>8 HBM + 8 HBF</b> | <b>Cache aware, 10%</b> | <b>10,419.17</b> | <b>2.640</b> | <b>6.88</b>        | <b>6.69</b>        | <b>96.46</b>   | <b>0.06</b>  | <b>14.82</b> |

relative to unbuffered cache-aware scheduling. The hybrid configurations retain most reusable prefixes on the device with the buffer enabled. Device hits include HBF residency and therefore do not imply HBM-speed access.

On  $H^3$ , adding the buffer reduces HBF KV writes by 69.33%, while completion time falls by 4.68%. On shared-site 4+4, writes fall by 91.96% and completion time by 20.21%. Thus, the buffer’s main benefit is reduced cache churn and write traffic. Shared-site 4+4 still finishes 1.84% later than buffered HBM-only serving, showing that greater cache retention does not always yield faster execution.

For HBF capacity  $C_F$ , endurance  $P$ , modeled write volume  $W_F$ , and workload duration  $T$ , we estimate

$$L_{\text{write}} = \frac{PC_F}{W_F/T}. \quad (8)$$

Assuming uniform wear, unit write amplification, and 100k P/E cycles, the reduced write rate raises estimated lifetime from 4.77 to 14.82 years for  $H^3$  and from 1.06 to 10.57 years for shared-site 4+4. These are extrapolations of the modeled average write rate. If commercial HBF endurance is lower, increasing admission headroom may reduce evictions and writes at the cost of a smaller active batch; the appropriate buffer depends on workload and device endurance.

### 5.3 Memory Architecture and Weight Placement

*Architecture.* Tables 5 and 6 compare the dense Llama and sparse-MoE GLM workloads. On the  $1\times$  trace,  $H^3$  with weights in HBM reduces completion time by 45.95% for Llama and 56.29% for GLM relative to HBM-only serving. Appropriately configured shared-site designs also provide substantial benefits with eight total stack sites: GLM 6+2 reduces completion time by 52.31% and energy by 9.09%.  $H^3$  TP16 with HBM weights nevertheless completes sooner, uses less energy, and has lower mean TPOT than this shared-site point. These comparisons evaluate complete configurations, including their placement policies and architecture-specific HBM/HBF service models.

Figure 6 visualizes completion time and modeled energy for six representative configurations from Table 6. The table retains

**Table 5: Llama-3.1-405B weight placement on 8 GPUs with eight-way tensor parallelism (TP8), the  $1\times$  trace (100k turns), and a 10% scheduling buffer.  $X + Y$  denotes  $X$  HBM and  $Y$  HBF stacks per GPU; 8+0 is HBM-only and  $H^3$  adds HBF stacks alongside HBM. Time is workload completion time; TPOT is mean per-request time per output token after the first. Arrows indicate lower is better; bold marks each column minimum.**

| Configuration          | Time (s) ↓    | Energy (MJ) ↓ | TPOT (ms) ↓  |
|------------------------|---------------|---------------|--------------|
| 8+0, HBM weights       | 39,666        | 53.60         | <b>90.85</b> |
| $H^3$ 8+8, HBM weights | 21,439        | <b>40.70</b>  | 438.61       |
| $H^3$ 8+8, HBF weights | <b>21,423</b> | 42.06         | 438.27       |

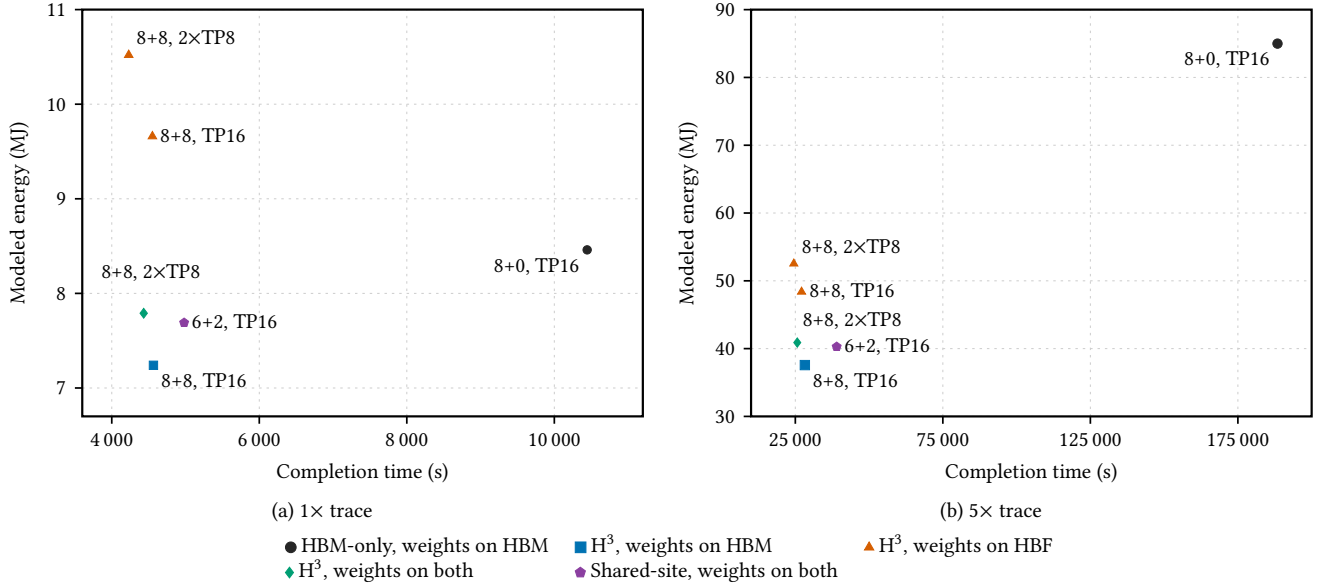
the complete comparison, including all shared-site allocations and TPOT.

*Weight placement and parallelism.* For Llama, moving weights to HBF changes completion time little but increases energy by 3.33%. For GLM TP16, it similarly offers little completion-time benefit while increasing energy by 33.42%, above the HBM-only baseline. Sparse attention reduces KV-read traffic, making weight placement more consequential for GLM’s energy budget.

The  $H^3$  split-TP8 configuration retains 160 logical experts plus 32 redundant copies in HBM per replica, with 96 experts in HBF and other weights in HBM. This leaves approximately 23.90 GB HBM per GPU for KV/index state. On the  $1\times$  trace, split placement reduces energy by 25.97% relative to all-HBF TP8 weights, at a 4.79% completion-time cost. Relative to TP16 with HBM weights, split TP8 is 2.89% faster but consumes 7.58% more energy. On the  $5\times$  trace, TP8 with all weights in HBF finishes fastest, 4.57% sooner than split TP8 but using 28.47% more energy; TP16 with HBM weights uses the least energy. The split configurations assume ideal EP balance and use average expert traffic to approximate rank service; this comparison jointly varies parallelism, placement, redundancy, and the expert-activity estimator.

**Table 6: GLM-5.2 architecture, placement, and parallelism on 16 GPUs with a 10% scheduling buffer.  $X + Y$  denotes  $X$  HBM and  $Y$  HBF stacks per GPU:  $H^3$  adds HBF alongside HBM, while shared-site designs divide eight stack sites between them. TP16 is one 16-GPU tensor-parallel replica;  $2 \times TP8$  is two 8-GPU replicas, using 16-way and 8-way expert parallelism, respectively, with data-parallel attention. HBM/HBF denotes all weights in that tier; Split places selected experts in HBM, remaining experts in HBF, and other weights in HBM, using offline expert lookup and ideal expert balance. The  $1 \times / 5 \times$  traces contain 100k/500k turns with original/extended contexts. Time is workload completion time; TPOT is mean per-request time per output token after the first. On the  $5 \times$  trace, among the tested shared-site allocations, 4+4 minimizes completion time and 6+2 minimizes energy. Bold marks each column minimum across all configurations. Figure 6 visualizes the completion-time and energy tradeoffs for six of these configurations.**

| Architecture | Stacks | Parallelism    | Weights | 1× trace     |             |               | 5× trace      |              |               |
|--------------|--------|----------------|---------|--------------|-------------|---------------|---------------|--------------|---------------|
|              |        |                |         | Time (s)     | Energy (MJ) | TPOT (ms)     | Time (s)      | Energy (MJ)  | TPOT (ms)     |
| HBM-only     | 8+0    | TP16           | HBM     | 10,445       | 8.46        | 234.09        | 188,425       | 84.99        | <b>216.31</b> |
| $H^3$        | 8+8    | TP16           | HBM     | 4,565        | <b>7.24</b> | 277.79        | 28,231        | <b>37.57</b> | 351.29        |
| $H^3$        | 8+8    | TP16           | HBF     | 4,551        | 9.66        | 276.42        | 27,117        | 48.38        | 332.42        |
| $H^3$        | 8+8    | $2 \times TP8$ | HBF     | <b>4,230</b> | 10.52       | <b>227.40</b> | <b>24,489</b> | 52.53        | 276.76        |
| $H^3$        | 8+8    | $2 \times TP8$ | Split   | 4,433        | 7.79        | 252.61        | 25,661        | 40.89        | 307.66        |
| Shared-site  | 2+6    | TP16           | Split   | 6,073        | 8.51        | 342.12        | 35,809        | 44.81        | 406.36        |
| Shared-site  | 4+4    | TP16           | Split   | 5,264        | 7.99        | 322.72        | 33,459        | 41.97        | 423.71        |
| Shared-site  | 6+2    | TP16           | Split   | 4,981        | 7.69        | 302.87        | 39,019        | 40.28        | 410.47        |



**Figure 6: Completion-time and modeled-energy tradeoffs for six GLM-5.2 configurations on (a) the  $1 \times$  and (b) the  $5 \times$  traces; see Table 6 for configuration details.**

*Workload and shared-site allocation.* The shared-site 2+6, 4+4, and 6+2 configurations retain 15+1, 100+32, and 144+32 logical experts plus redundant copies in HBM, respectively. Each has 32 redundant copies in total; remaining expert copies reside in HBF and other weights remain in HBM. HBM experts are selected by training-profile frequency. The HBM-only and all-HBM-weight  $H^3$  references use the original expert-activity model without redundant copies.

On the  $1 \times$  trace, 6+2 provides the lowest completion time, energy, and mean TPOT among the tested shared-site allocations. On the  $5 \times$  trace, 4+4 completes 14.25% sooner than 6+2, while 6+2 consumes 4.01% less energy than 4+4. Longer contexts increase KV-space requirements, but expert placement also determines how much HBM remains available to KV. With 144+32 HBM experts, 6+2 achieves 98.11% device hits on the extended trace. The preferred allocation therefore depends on workload and whether completion

**Table 7: Energy-savings contributions relative to same-model, same-workload HBM-only energy. Llama denotes Llama-3.1-405B; GLM denotes GLM-5.2.  $X + Y$  gives HBM+HBF stack counts per GPU;  $1\times/5\times$  denotes the original/extended trace with 100k/500k turns. The 8+8 columns use  $H^3$  with HBM weights; 6+2 and 4+4 use shared-site expert placement. Entries are percentage points: positive values save energy and negative values add cost. KV denotes key-value cache; avoided prefill counts computation only. Other savings are the residual; bold totals may differ due to rounding.**

| Contribution              | Llama<br>8+8<br>1× | GLM<br>6+2<br>1× | GLM<br>8+8<br>1× | GLM<br>4+4<br>5× | GLM<br>8+8<br>5× |
|---------------------------|--------------------|------------------|------------------|------------------|------------------|
| Reduced weight reads      | +15.22             | +7.52            | +12.70           | +47.15           | +51.22           |
| Increased KV/index reads  | -13.16             | -0.75            | -0.60            | -1.51            | -0.50            |
| Avoided repeated prefill  | +20.25             | 0.00             | 0.00             | 0.00             | 0.00             |
| Other net savings         | +1.74              | +2.31            | +2.31            | +4.97            | +5.07            |
| <b>Net energy savings</b> | <b>24.06%</b>      | <b>9.09%</b>     | <b>14.41%</b>    | <b>50.62%</b>    | <b>55.80%</b>    |

time or energy is the objective.  $H^3$  retains the strongest completion-time results in these comparisons.

Larger batches also introduce a latency tradeoff: mean TPOT rises from 90.85 to 438.61 ms for Llama and from 234.09 to 277.79 ms for GLM  $H^3$  with HBM weights on the  $1\times$  trace. On the  $5\times$  trace, every evaluated hybrid configuration has higher mean TPOT than HBM-only serving. Our high-throughput evaluation therefore does not imply improved per-request latency.

#### 5.4 Sources of Energy Savings

Table 7 separates energy savings from weight reuse, KV/index reads, and avoided prefill computation. On the  $1\times$  trace,  $H^3$  with HBM weights saves 24.06% of baseline energy for Llama and 14.41% for GLM. Dense serving achieves greater percentage savings here despite its larger HBF KV-read cost: prefix reuse avoids 53.33 million query-token computations, contributing 20.25 percentage points of baseline energy savings. The compared GLM runs perform the same prefill work without recomputation.

Batching amortizes weight loads and reduces kernel launches. Llama’s weight-read energy falls by 77.8%, compared with 18.0% for GLM shared-site 6+2 and 30.5% for GLM  $H^3$  8+8. Weight reuse and reduced recomputation lower modeled energy, whereas kernel saturation reduces latency. Rank-local DPA batches and aggregate MoE batches differ in their opportunities for weight reuse.

For  $H^3$  GLM with HBM weights, repricing HBF reads from twice to four times HBM read energy at fixed traffic retains 13.09% energy savings, compared with 14.41%. Most read energy in this configuration comes from HBM-resident weights, limiting sensitivity to HBF read energy. This result is placement-specific: shared-site 6+2 also reads expert weights from HBF.

#### 5.5 Scope and Limitations

We evaluate high-throughput serving; latency-constrained serving, where small batches limit the benefit of added capacity, is outside the scope of this study. We do not model a production HBF controller, complete package power, wear leveling, or every

activation and staging write. Kernel fits and the layer-class approximation limit extrapolation beyond the profiled hardware. Repeating sessions increases offered work but does not add independently collected conversations. Saved runs check completion counts, queue drainage, token conservation, tier accounting, and reproducibility. We use ordinary autoregressive decoding without speculative decoding.

## 6 Conclusion

For high-throughput LLM serving, high-bandwidth flash can turn added capacity into faster, more energy-efficient execution, but only when data placement and scheduling account for its costs. Combining an HBM-HBF-host hierarchy with buffered cache-aware scheduling, HBF-augmented systems with weights in HBM reduce workload completion time by 46–56% and modeled energy by 14–24% relative to HBM-only serving on the  $1\times$  traces for Llama-3.1-405B and GLM-5.2. On the GLM-5.2  $5\times$  trace, the HBM-weight configuration achieves a  $6.7\times$  speedup and 56% energy savings. These gains come mainly from retaining reusable KV state on the device and amortizing weight reads over larger batches. Write endurance, often cited as the key obstacle to placing KV cache in flash, becomes manageable once admission is controlled: reserving 10% headroom reduces HBF KV writes by 69% and extends estimated write lifetime from 4.8 to 14.8 years. HBF should therefore be co-designed with the scheduler rather than treated as passive overflow capacity.

## References

- [1] Keivan Alizadeh, Seyed Iman Mirzadeh, Dmitry Belenko, S. Khatamifard, Minsik Cho, Carlo C Del Mundo, Mohammad Rastegari, and Mehrdad Farajtabar. 2024. LLM in a flash: Efficient Large Language Model Inference with Limited Memory. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 12562–12584. doi:10.18653/v1/2024.acl-long.678
- [2] Yushi Bai, Qian Dong, Ting Jiang, Xin Lv, Zhengxiao Du, Aohan Zeng, Jie Tang, and Juanzi Li. 2026. IndexCache: Accelerating Sparse Attention via Cross-Layer Index Reuse. arXiv:2603.12201 doi:10.48550/arXiv.2603.12201
- [3] Jaehong Cho, Hyunmin Choi, Guseul Heo, and Jongse Park. 2026. LLMservingSim 2.0: A Unified Simulator for Heterogeneous and Disaggregated LLM Serving Infrastructure. In *2026 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 1–14. doi:10.1109/ispass69572.2026.00012
- [4] DeepSeek-AI et al. 2024. DeepSeek-V3 Technical Report. arXiv:2412.19437 doi:10.48550/arXiv.2412.19437
- [5] Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W. Mahoney, and Kurt Keutzer. 2024. AI and Memory Wall. *IEEE Micro* 44, 3 (2024), 33–39. doi:10.1109/mm.2024.3373763
- [6] Minh Ho, Euseok Kim, and Hoshik Kim. 2026.  $H^3$ : Hybrid Architecture Using High Bandwidth Memory and High Bandwidth Flash for Cost-Efficient LLM Inference. *IEEE Computer Architecture Letters* 25, 1 (2026), 49–52. doi:10.1109/lca.2026.3660969
- [7] Po-Kai Hsu, Weihong Xu, Qunyou Liu, Tajana Rosing, and Shimeng Yu. 2026. HAVEN: High-Bandwidth Flash Augmented Vector Engine for Large-Scale Approximate Nearest-Neighbor Search Acceleration. arXiv preprint. arXiv:2603.01175 <https://arxiv.org/abs/2603.01175v1> Version 1, revised 2026-03-01.
- [8] Inho Jeong, Sunghyeon Woo, Sol Namkung, and Dongsuk Jeon. 2025. HiFC: High-efficiency Flash-based KV Cache Swapping for Scaling LLM Inference. In *Advances in Neural Information Processing Systems*, Vol. 38, Main Conference. Curran Associates, Inc., 47561–47590. doi:10.52202/085713-1587
- [9] Hao Kang, Ziyang Li, Weili Xu, Xinyu Yang, Yinfang Chen, Junxiong Wang, Beidi Chen, Tushar Krishna, Chenfeng Xu, and Simran Arora. 2026. ThunderAgent: A Simple, Fast and Program-Aware Agentic Inference System. arXiv preprint. arXiv:2602.13692 <https://arxiv.org/abs/2602.13692v3> Version 3, revised 2026-06-30.
- [10] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention.

- In *Proceedings of the 29th Symposium on Operating Systems Principles*. ACM, 611–626. doi:10.1145/3600006.3613165
- [11] Lenovo. [n. d.]. ThinkSystem NVIDIA HGX B200 180GB 1000W GPU Product Guide. Lenovo Press. <https://lenovopress.lenovo.com/lp2226-thinksystem-nvidia-b200-180gb-1000w-gpu>
  - [12] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2021. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=qrwe7XHTmYb>
  - [13] Hanchen Li, Runyuan He, Qiuyang Mang, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Huanchen Zhang, Alvin Cheung, Joseph Gonzalez, and Ion Stoica. 2025. Continuum: Efficient and Robust Multi-Turn LLM Agent Scheduling with KV Cache Time-to-Live. arXiv preprint. arXiv:2511.02230 <https://arxiv.org/abs/2511.02230v7> Version 7, revised 2026-09-08.
  - [14] Zhuoran Li, Zhuohang Bian, Xin Huang, Yibo Zhao, Guangyu Sun, and Youwei Zhuo. 2026. HBF Sucks? A Full-Stack Characterization of High-Bandwidth Flash for KV-Centric LLM Serving. arXiv preprint. arXiv:2608.11668 <https://arxiv.org/abs/2608.11668v3> Version 3, revised 2026-08-25.
  - [15] Yuhao Liu, Yihua Cheng, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Rui Zhang, Kuntai Du, and Junchen Jiang. 2025. LMCACHE: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference. arXiv preprint. arXiv:2510.09665 <https://arxiv.org/abs/2510.09665v2> Version 2, revised 2025-12-05.
  - [16] Yuhao Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. 2024. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*. ACM, 38–56. doi:10.1145/3651890.3672274
  - [17] LMCACHE. 2026. LMCACHE Agentic Dataset: Multi-Turn LLM Agent Sessions for KV Cache Benchmarking. <https://huggingface.co/datasets/sammshen/lmcache-agentic-traces>
  - [18] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E. Gonzalez, and Ion Stoica. 2025. Autellix: An Efficient Serving Engine for LLM Agents as General Programs. arXiv preprint. arXiv:2502.13965 <https://arxiv.org/abs/2502.13965v1> Version 1, revised 2025-02-19.
  - [19] Xiaoyu Ma and David Patterson. 2026. Challenges and Research Directions for Large Language Model Inference Hardware. *Computer* 59, 5 (2026), 55–64. doi:10.1109/mc.2026.3652916
  - [20] Micron Technology. [n. d.]. HBM3E. <https://www.micron.com/products/memory/hbm/hbm3e>
  - [21] Ki-Il Moon, Ho-Young Son, and Kangwook Lee. 2023. Advanced Packaging Technologies in Memory Applications for Future Generative AI Era. In *2023 International Electron Devices Meeting (IEDM)*. IEEE, 1–4. doi:10.1109/IEDM45741.2023.10413890
  - [22] Vinicius Petrucci, Felipe Zacarias, and Vishal Tanna. 2026. Is High-Bandwidth Flash All You Need?. In *3rd Workshop on Hot Topics in System Infrastructure (HotInfra)*. Raleigh, NC, USA. <https://hotinfra.org/2026/papers/hotinfra26-final83.pdf> Workshop co-located with ISCA 2026, June 28, 2026.
  - [23] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation – A KVCache-centric Architecture for Serving LLM Chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. USENIX Association, 155–170. <https://www.usenix.org/conference/fast25/presentation/qin>
  - [24] Sandisk. 2025. *Sandisk Unveils the Future of Memory Architecture for AI: Introducing High Bandwidth Flash*. HBF Fact Sheet / Tech Brief. Sandisk. [https://documents.sandisk.com/content/dam/asset-library/en\\_us/assets/public/sandisk/collateral/company/Sandisk-HBF-Fact-Sheet.pdf](https://documents.sandisk.com/content/dam/asset-library/en_us/assets/public/sandisk/collateral/company/Sandisk-HBF-Fact-Sheet.pdf)
  - [25] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Re, Ion Stoica, and Ce Zhang. 2023. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*. PMLR, 31094–31116. <https://proceedings.mlr.press/v202/sheng23a.html>
  - [26] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. arXiv:1909.08053 doi:10.48550/arXiv.1909.08053
  - [27] Dowon Son, Yonggon Park, Hyunuk Cho, Hyungkyu Ham, Onur Mutlu, Sungjin Lee, Gwangsun Kim, and Jisung Park. 2026. Exploring High-Bandwidth Flash for Modern LLM Inference: Opportunities and Challenges. *IEEE Computer Architecture Letters* 25, 2 (2026), 251–254. doi:10.1109/LCA.2026.3705817
  - [28] TSMC. 2020. TSMC Showcases Leading Technologies at Online Technology Symposium and OIP Ecosystem Forum. <https://pr.tsmc.com/english/news/2729>
  - [29] TSMC. 2021. TSMC Expands Advanced Technology Leadership with N4P Process. <https://pr.tsmc.com/english/news/2874>
  - [30] William Won, Taekyung Heo, Saeed Rashidi, Srinivas Sridharan, Sudarshan Srinivasan, and Tushar Krishna. 2023. ASTRA-sim2.0: Modeling Hierarchical Networks and Disaggregated Systems for Large-model Training at Scale. In *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 283–294. doi:10.1109/ISPASS57527.2023.00035
  - [31] Haoran Wu, Zeyu Cao, Yao Lai, Binglei Lou, Jiayi Nie, Can Xiao, Timi Adeniran, Przemyslaw Forys, Kauser Johar, Catriona Wright, Junyi Liu, Kai Shi, Nicholas D. Lane, Rika Antonova, Jianyi Cheng, Timothy Jones, Aaron Zhao, and Robert Mullins. 2026. MemExplorer: Navigating the Heterogeneous Memory Design Space for Agentic Inference NPUs. arXiv preprint. arXiv:2604.16007 <https://arxiv.org/abs/2604.16007v1> Version 1, revised 2026-04-17.
  - [32] Zhiqiang Xie. 2025. SGLang HiCache: Fast Hierarchical KV Caching with Your Favorite Storage Backends. LMSYS Org blog. <https://www.lmsys.org/blog/2025-09-10-sglang-hicache/> Published September 10, 2025. Accessed September 11, 2026.
  - [33] Jiayi Yao, Hanchen Li, Yuhao Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. In *Proceedings of the Twentieth European Conference on Computer Systems*. ACM, 94–109. doi:10.1145/3689031.3696098
  - [34] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, 521–538. <https://www.usenix.org/conference/osdi22/presentation/yy>
  - [35] Sebastian Zhao, Minseo Kim, Coleman Richard Charles Hooper, Luca Manolache, Michael W. Mahoney, Sophia Shao, Kurt Keutzer, and Amir Gholami. 2026. LLM Inference in a Flash!. In *Machine Learning for Computer Architecture and Systems 2026*. <https://openreview.net/forum?id=gSphYexssc> ISCA 2026 workshop, oral presentation.
  - [36] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., 62557–62583. doi:10.52202/079017-2000